Chapitre 2 — Types, variables, entrées-sorties et opérateurs

Prérequis : chapitre 1 · **Exercices :** `E02_Types_variables_entrees_sorties`

Objectifs

- déclarer et initialiser des variables de types adaptés, utiliser des constantes ;
- afficher des données et lire des saisies au clavier, y compris une ligne complète ;
- manipuler des chaînes de caractères ;
- utiliser les opérateurs arithmétiques sans erreur de division.

Sommaire

- Chapitre 2 — Types, variables, entrées-sorties et opérateurs
  - Objectifs
  - Sommaire
  - 1. Les variables
    - 1.1 Déclaration et initialisation
  - 2. Les constantes
  - 3. Afficher avec `std::cout`
  - 4. Les chaînes `std::string`
  - 5. Lire au clavier avec `std::cin`
    - 5.1 Lire une chaîne sans espaces
    - 5.2 Lire une ligne complète
    - 5.3 Enchaîner `>>` et `std::getline`
  - 6. Les opérateurs arithmétiques
    - 6.1 Division entière et division réelle
    - 6.2 Affectations abrégées
  - Erreurs fréquentes
  - Synthèse

1. Les variables

Une **variable** est une donnée en mémoire, identifiée par un **nom** et associée à un **type**, qui détermine les valeurs possibles et les opérations autorisées.

Un nom contient des lettres, des chiffres et `_`, ne commence pas par un chiffre et n'est pas un mot-clé. On choisit des noms explicites en `camelCase` (`nombreCapteurs`), sans accents, et on évite de commencer par `_` (formes réservées).

Type	Utilisation	Exemple de valeur
`bool`	Valeur logique	`true`, `false`
`char`	Caractère (un octet)	`'A'`
`int`	Entier signé (le plus souvent 32 bits)	`-25`
`unsigned int`	Entier non signé	`25u`
`double`	Nombre à virgule flottante	`12.5`
`std::string`	Chaîne de caractères (`#include <string>`)	`"Bonjour"`

1.1 Déclaration et initialisation

La **déclaration** crée la variable. L'**initialisation** lui donne sa première valeur.

```
int age{20};
double temperature{21.5};
bool systemeActif{true};
std::string nom{"Carnus"};
int compteur{};          // accolades vides : 0 (false, chaine vide...)
```

L'initialisation avec accolades **refuse les conversions qui perdent de l'information** :

```
// int valeur{3.7};   // erreur de compilation : perte de la partie decimale
int valeur = 3.7;     // compile, mais valeur vaut 3 : perte silencieuse
```

> **Remarque :** une variable locale de type simple déclarée **sans** initialisation (`int nombre;`) n'a pas de valeur définie. La lire est un **comportement indéfini** : le programme peut afficher n'importe quoi, ou se comporter différemment d'une exécution à l'autre. On initialise donc **toujours**.

2. Les constantes

```
const int secondesParMinute{60};          // non modifiable apres initialisation
constexpr int SECONDES_PAR_HEURE{3600};   // valeur connue a la compilation
```

`constexpr` est le choix adapté aux constantes fixes d'un programme. On rencontre aussi des macros (`#define SECONDES 60`) ; non typées, on leur préfère `const` ou `constexpr`.

3. Afficher avec `std::cout`

```cpp
#include <iostream>

int main() {
    int age{20};
    bool actif{true};

    std::cout << "Age : " << age << " ans" << '\n';
    std::cout << actif << ' ' << std::boolalpha << actif << '\n';

    return 0;
}
```

Résultat :

```
Age : 20 ans
1 true
```

- Séquences d'échappement : `\n` (retour à la ligne), `\t` (tabulation), `\\`, `\"`, `\'`.
- `std::endl` insère une fin de ligne **et** vide le tampon d'affichage ; pour un affichage ordinaire, `'\n'` suffit.
- Un `bool` s'affiche `1` ou `0` ; `std::boolalpha` affiche `true` ou `false` et reste actif sur le flux.

4. Les chaînes `std::string`

```cpp
#include <iostream>
#include <string>

int main() {
    std::string prenom{"Charles"};
    std::string nom{"Carnus"};
    std::string identite{prenom + " " + nom};   // concatenation

    std::cout << identite << '\n';
    std::cout << "Longueur : " << identite.length() << '\n';
    std::cout << "Initiale : " << identite[0] << '\n';

    std::string x{"été"};
    std::cout << "Longueur de x est : " << x.length() << '\n';

    return 0;
}
```

Résultat :

```
Charles Carnus
Longueur : 14
Initiale : C
Longueur de x est : 5
```

- Les indices commencent à **0** : le dernier caractère a l'indice `length() - 1`.
- `[]` ne vérifie pas l'indice (hors limites : comportement indéfini).

- `length()` (ou `size()`) compte des `char` : pour un texte accentué en UTF-8, le résultat peut dépasser le nombre de caractères visibles.

5. Lire au clavier avec `std::cin`

`std::cin >> variable` lit une valeur selon le **type** de la variable, en s'arrêtant au premier espace. Pour lire une **ligne complète**, on utilise `std::getline(std::cin, texte)`.

5.1 Lire une chaîne sans espaces

```
std::string prenom{};

std::cout << "Prenom : ";
std::cin >> prenom;
```

Si l'utilisateur saisit `Charles Carnus`, seule la première partie, `Charles`, est lue dans `prenom`.

5.2 Lire une ligne complète

Pour lire une ligne entière, y compris les espaces, on utilise `std::getline`.

```
#include <iostream>
#include <string>

int main() {
    std::string nomComplet{};

    std::cout << "Nom complet : ";
    std::getline(std::cin, nomComplet);

    std::cout << "Bonjour " << nomComplet << '\n';

    return 0;
}
```

5.3 Enchaîner `>>` et `std::getline`

`std::cin >> age` lit le nombre mais laisse dans le flux le **retour à la ligne** tapé par l'utilisateur. Un `std::getline` placé juste après lit alors ce retour à la ligne… et renvoie une chaîne **vide**, sans laisser l'utilisateur saisir.

La solution : `std::cin >> std::ws` saute les espaces et retours à la ligne restants avant la lecture de la ligne.

```cpp
#include <iostream>
#include <string>

int main() {
    int age{};
    std::string nomComplet{};

    std::cout << "Age : ";
    std::cin >> age;                                    // le retour a la ligne reste dans le
flux

    std::cout << "Nom complet : ";
    std::getline(std::cin >> std::ws, nomComplet);    // std::ws saute les blancs restants

    std::cout << nomComplet << ", " << age << " ans\n";

    return 0;
}
```

Exemple d'exécution (les saisies suivent les deux-points) :

```
Age : 20
Nom complet : Charles Carnus
Charles Carnus, 20 ans
```

> **Rappel :** ces lectures supposent que l'utilisateur saisit des données correctes. Le contrôle des saisies est étudié au chapitre 3 (§4) et au chapitre 4 (§6).

6. Les opérateurs arithmétiques

Opérateur	Opération	Exemple
`+`	Addition	`a + b`
`-`	Soustraction	`a - b`
`*`	Multiplication	`a * b`
`/`	Division	`a / b`
`%`	Reste de la division entière	`a % b`

La multiplication et la division sont évaluées avant l'addition et la soustraction. En cas de doute, on ajoute des parenthèses : `(a + b) / 2`.

6.1 Division entière et division réelle

Entre **deux entiers**, `/` est une division **entière** : la partie décimale est supprimée.

```cpp
#include <iostream>

int main() {
    int a{7};
    int b{3};

    std::cout << a / b << '\n';                      // division entiere
    std::cout << a % b << '\n';                      // reste
    std::cout << static_cast<double>(a) / b << '\n';  // division reelle

    return 0;
}
```

Résultat :

```
2
1
2.33333
```

Un seul opérande réel suffit. En revanche, `static_cast<double>(a / b)` vaut `2` : la division entière est faite **avant** la conversion.

Le reste `%` sert à tester la parité (`n % 2 == 0`), à découper une durée ou à revenir au début d'un cycle (`i % 4`, chapitre 6).

6.2 Affectations abrégées

Forme complète	Forme abrégée
`a = a + b;`	`a += b;`
`a = a - b;`	`a -= b;`
`a = a * b;`	`a *= b;`
`a = a / b;`	`a /= b;`
`a = a % b;`	`a %= b;`
`a = a + 1;`	`a++;` ou `++a;`
`a = a - 1;`	`a--;` ou `--a;`

Utilisées seules, `a++;` et `++a;` font la même chose. Dans une expression, elles diffèrent : `x = a++;` copie l'**ancienne** valeur de `a`, alors que `x = ++a;` copie la **nouvelle**.

> **Conseil :** pour éviter toute ambiguïté, on incrémente sur une ligne séparée.

Erreurs fréquentes

Erreur	Effet	Correction
Variable non initialisée	Valeur imprévisible (comportement indéfini)	Initialiser à la déclaration : `int n{};`
`int valeur = 3.7;`	Partie décimale perdue sans message	Initialiser avec `{}` (le compilateur refuse) et choisir `double`
`7 / 2` pour obtenir 3,5	Division entière : résultat 3	`static_cast<double>(a) / b`
`static_cast<double>(a / b)`	Conversion faite trop tard : résultat entier	Convertir un opérande **avant** la division
`std::getline` juste après `std::cin >>`	Chaîne vide, saisie « sautée »	`std::getline(std::cin >> std::ws, texte)`
`std::cin >> texte` pour un nom avec espace	Seul le premier mot est lu	`std::getline`
`texte[texte.length()]`	Hors limites : comportement indéfini	Indices de `0` à `length() - 1`
`bool` affiché `1` ou `0`	Affichage peu lisible	`std::boolalpha`

Synthèse

- Une variable a un **nom** et un **type** ; on l'initialise toujours, de préférence avec `{}`, qui refuse les pertes d'information.
- Constantes : `constexpr` pour une valeur fixe connue à la compilation, `const` pour une valeur non modifiable.
- `std::cout << …` affiche ; `'\n'` termine la ligne ; `std::boolalpha` affiche les booléens en toutes lettres.
- `std::string` : concaténation avec `+`, `length()`, indices de `0` à `length() - 1`.
- `std::cin >> x` lit un mot ou une valeur ; `std::getline(std::cin, ligne)` lit une ligne ; après un `>>`, on écrit `std::getline(std::cin >> std::ws, ligne)`.
- Entre deux entiers, `/` est une division entière et `%` donne le reste ; pour une division réelle, on convertit un opérande **avant** la division.